Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mbead functions

Synopsis

The functions prefixed with Mbead make up the Bead module. The Bead module allows you to measure and verify beads, such as strips of glue or lead. For example, you can verify that a bead of glue occurs at a required position, that its width falls within a certain range, that it does not have any missing sections (gaps), and that its gaps do not exceed a specified size.

Functions

MbeadAlloc
MbeadAllocResult
MbeadControl
MbeadDraw
MbeadFree
MbeadGetNeighbors
MbeadGetResult
MbeadInquire
MbeadRestore
MbeadSave
MbeadStream
MbeadTemplate
MbeadTrain
MbeadVerify

MbeadAlloc

Synopsis

Allocate a bead context.

Syntax

```
AIL_ID MbeadAlloc(
    AIL_ID SysId,            //in
    AIL_INT64 Strategy,      //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *ContextBeadIdPtr //out
)
```

Description

This function allocates a bead context on the specified system.

Any value that you provide to the Aurora Imaging Library Bead module (for example, position and width) must be specified in appropriate units (pixel or real-world), depending on whether your images are calibrated. For more information, see the Camera calibration - overview section of Chapter 28: Calibrating your camera setup. Note that, if you are specifying pixel units, make sure the pixel sizes are consistent across all camera calibrations (for example, pixel sizes should be the same in the training and verification images). When using different camera calibration contexts, it is recommended that you use world units.

When the bead context is no longer required, release it using MbeadFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the bead context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the bead context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Strategy

Specifies the strategy with which to extract and analyze beads. This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the strategy																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default strategy.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextBeadIdPtr

Specifies the address of the variable in which to write the bead context identifier or specifies the data type that the function should use to return the bead context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated bead context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated bead context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BEAD_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the bead context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated bead context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the bead context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BEAD_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadAllocResult

Synopsis

Allocate a bead result buffer.

Syntax

```
AIL_ID MbeadAllocResult(
    AIL_ID SysId,           //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *ResultBeadIdPtr //out
)
```

Description

This function allocates a bead result buffer, on the specified system, to store the results from an MbeadVerify() operation.

When the bead result buffer is no longer required, release it using MbeadFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the bead result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the result buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultBeadIdPtr

Specifies the address of the variable in which to write the bead result buffer identifier or specifies the data type that the function should use to return the bead result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated bead result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated bead result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BEAD_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the bead result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated bead result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the bead result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BEAD_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadControl

Synopsis

Control a bead context or template setting.

Syntax

```
void MbeadControl(
    AIL_ID ContextBeadId,   //out
    AIL_INT LabelOrIndex,   //in
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control the specified setting for a bead context, or for the templates contained within the context. You can use these settings to manage the training (MbeadTrain()) and verification phases (MbeadVerify()). You can inquire most of these settings using MbeadInquire().

Settings for the training phase have an indirect affect on the verification phase, since it uses trained templates. Therefore if you modify a training phase setting, or a setting that applies to both the training and verification phases, you must re-call MbeadTrain() before the next call to MbeadVerify(). Settings that are exclusively for the verification phase are not used by, and have no effect on, the training phase; if you modify such settings, you need not re-call MbeadTrain().

This function's settings affect either the template's training status (established after calling MbeadTrain()) or verification status (established after calling MbeadVerify()). Depending on the status, you might have to adjust the related settings. To retrieve information about the training status, use MbeadInquire() with M_STATUS; to retrieve information about the verification status, use MbeadGetResult() with M_STATUS.

You can also use this function to control operations with MbeadGetNeighbors().

If you are specifying pixel units, make sure the pixel sizes are consistent across all camera calibrations (for example, pixel sizes should be the same in the training and verification images). When using different camera calibration contexts, it is recommended that you use world units.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MbeadControl()** internally calls the *AIL_DOUBLE* version of this function (MbeadControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MbeadControlInt64()).

Parameters

ContextBeadId

Specifies the identifier of the bead context to control. The bead context must have been previously allocated on the required system using MbeadAlloc().

LabelOrIndex

Specifies the bead template (one or all) to control, or specifies that you are controlling a global setting of the bead context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to control																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of an existing template to control.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index. The index must be greater than or equal to 0.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of an existing template to control.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label. The label must be greater than 0.															
M_ALL	Applies the specified control setting to all the templates contained within the bead context. In this case, the control type setting must be supported by all the templates.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Controls a global setting of a bead context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- To control bead templates for the training phase
- To control measured beads for the verification phase

- To control how Aurora Imaging Library extracts beads from an image for the training and verification phases
- To associate a camera calibration context with a training image
- To control operations with MbeadGetNeighbors()

The following ControlType and corresponding ControlValue parameter settings are used to control templates (one or all) in a bead context for the training phase (MbeadTrain()). In this case, you must set the LabelOrIndex parameter to a specific template (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()) or to all templates (M_ALL).

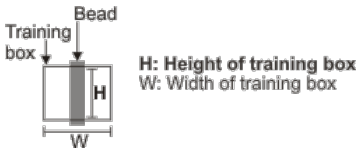
Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control bead templates for the training phase																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_INTENSITY_NOMINAL_MODE		Sets how Aurora Imaging Library establishes the nominal pixel intensity of the template. You can use the pixel intensity of the template when verifying beads. That is, Aurora Imaging Library can validate the brightness of each measured point according to the brightness of its associated trained point. Aurora Imaging Library calculates the intensity for each point at the center of its corresponding bead width. You can only use this value with stripe-beads. INQ															
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to establish the nominal pixel intensity from the training image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_INTENSITY_NOMINAL_MODE has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that you will explicitly set the pixel intensity, using M_INTENSITY_NOMINAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_CENTER_X		Sets the X-coordinate of the center of a template whose path follows a circle. This value only applies if M_TRAINING_PATH is set to M_CIRCLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b

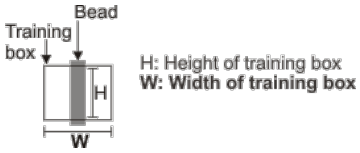
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the center's X-coordinate, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_CENTER_Y		Sets the Y-coordinate of the center of a template whose path follows a circle. This value only applies if M_TRAINING_PATH is set to M_CIRCLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the center's Y-coordinate, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_RADIUS		Sets the radius of a template whose path follows a circle. This value only applies if M_TRAINING_PATH is set to M_CIRCLE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the radius, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_INPUT_UNITS		Sets the units with which to interpret the M_CLOSEST_POINT_MAX_DISTANCE, M_FAIL_WARNING_OFFSET, M_GAP_MAX_LENGTH, M_OFFSET_MAX, M_TEMPLATE_CIRCLE_CENTER_X, M_TEMPLATE_CIRCLE_CENTER_Y, M_TEMPLATE_CIRCLE_RADIUS, M_TEMPLATE_SEGMENT_END_X, M_TEMPLATE_SEGMENT_END_Y, M_TEMPLATE_SEGMENT_START_X, M_TEMPLATE_SEGMENT_START_Y, M_WIDTH_DELTA_NEG, M_WIDTH_DELTA_POS, and M_WIDTH_NOMINAL control types, as well as the units with which to interpret the template's position and dimension values specified using MbeadTemplate(). This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling MbeadDraw() to draw features of the template, or calling MbeadTrain() or MbeadVerify() generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_END_X		Sets the X-coordinate of the end of a template whose path follows a segment. This value only applies if M_TRAINING_PATH is set to M_SEGMENT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the end of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_SEGMENT_END_Y	Sets the Y-coordinate of the end of a template whose path follows a segment. This value only applies if M_TRAINING_PATH is set to M_SEGMENT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the end of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_START_X	Sets the X-coordinate of the start of a template whose path follows a segment. This value only applies if M_TRAINING_PATH is set to M_SEGMENT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate of the start of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_START_Y	Sets the Y-coordinate of the start of a template whose path follows a segment. This value only applies if M_TRAINING_PATH is set to M_SEGMENT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate of the start of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINING_BOX_HEIGHT		Sets the height of the training box in which to establish the trained points.  When you call MbeadTrain(), Aurora Imaging Library uses the specified height of the training box to achieve the best height value based on all your settings (for example, if you use a training image). To inquire the actual height Aurora Imaging Library establishes (the trained height), use MbeadInquire() with M_TRAINED_BOX_HEIGHT. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the height, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_BOX_INPUT_UNITS		Sets the units with which to interpret the M_TRAINING_BOX_HEIGHT, M_TRAINING_BOX_WIDTH, M_TRAINING_BOX_SPACING, and M_TRAINING_WIDTH_NOMINAL control types. This essentially sets the input coordinate system to use. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. If world units are specified, calling MbeadDraw() to draw features of the template, or calling MbeadTrain() or MbeadVerify(), generates an error if the operation is not performed on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINING_BOX_SPACING		Sets the distance between the center of the training boxes in which to establish the trained points. Since Aurora Imaging Library establishes trained points at regular intervals along the template's path, the specified spacing is adjusted accordingly. To inquire the actual spacing Aurora Imaging Library uses, call MbeadInquire() with M_TRAINED_BOX_SPACING. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_BOX_SPACING has no effect. If a template's path follows a fixed circle or a segment (M_TRAINING_PATH set to M_CIRCLE or M_SEGMENT) and you disable spacing, Aurora Imaging Library internally creates the minimum number of critical points required to geometrically establish the path. If a template's path follows a fixed polyline (M_POLYLINE) and you disable spacing, the trained points will be the same as the vertices specified when adding or modifying the template (MbeadTemplate()). Such vertices can, for example, come from a CAD file, where the exact path of the polyline is known (but not necessarily the width). If a template's path follows a polyline that is refined by a training image (M_POLYLINE_SEED) and you disable spacing, the number of trained points will be the same as the number of vertices specified when adding or modifying the template (MbeadTemplate()), however Aurora Imaging Library will reposition them accordingly (with respect to the training image).	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the spacing, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINING_BOX_WIDTH		Sets the width of the training box in which to establish the trained points.  When you call MbeadTrain(), Aurora Imaging Library uses the specified width of the training box to achieve the best width value based on all your settings (for example, if you use a training image). To inquire the actual width Aurora Imaging Library establishes (the trained width), use MbeadInquire() with M_TRAINED_BOX_WIDTH. Note that Aurora Imaging Library does not typically alter the specified training box width. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 200.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the width, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_PATH		Sets how Aurora Imaging Library establishes the path of the bead template. Along the path, Aurora Imaging Library establishes trained points at regular intervals based on the specified spacing (M_TRAINING_BOX_SPACING). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_POLYLINE_SEED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CIRCLE	Specifies that the path of the bead template follows a fixed circle defined with M_TEMPLATE_CIRCLE_CENTER_X, M_TEMPLATE_CIRCLE_CENTER_Y, and M_TEMPLATE_CIRCLE_RADIUS. The trained position of the path always follows the exact position of the specified circle. If the template contains vertices (MbeadTemplate() with M_ADD or M_INSERT), Aurora Imaging Library ignores them.	c		e	f	g	a		i		j	k	l	m	n	b

	M_POLYLINE	Specifies that the path of the bead template follows a fixed polyline. The trained position of the path always follows the exact position of the specified polyline. To specify the polyline, you must set its vertices when adding the template using MbeadTemplate(). You can also modify the vertices of the polyline with MbeadTemplate() operations such as M_DELETE and M_INSERT. Note that with MbeadTemplate(), you can also specify the vertices of a polyline (or polygon) from a 2D graphics list. A polygon is simply a polyline that Aurora Imaging Library automatically closes by connecting the final vertex to the first with a straight line. Any operation or setting that applies to polylines also applies to polygons.	c		e	f	g	a		i		j	k	l	m	n	b
	M_POLYLINE_SEED	Specifies that the path of the bead template follows a polyline that is refined by the training image set with MbeadTrain(). To define the polyline, you must specify its vertices when adding the template using MbeadTemplate() with M_ADD. You can also modify the vertices of a polyline with MbeadTemplate() operations such as M_DELETE and M_INSERT. Note that with MbeadTemplate(), you can also specify the vertices of a polyline (or polygon) from a 2D graphics list. A polygon is simply a polyline that Aurora Imaging Library automatically closes by connecting the final vertex to the first with a straight line. Any operation or setting that applies to polylines also applies to polygons.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEGMENT	Specifies that the path of the bead template follows a fixed segment defined with M_TEMPLATE_SEGMENT_START_X, M_TEMPLATE_SEGMENT_START_Y, M_TEMPLATE_SEGMENT_END_X, and M_TEMPLATE_SEGMENT_END_Y. The trained position of the path always follows the exact position of the specified segment. If the template contains vertices (MbeadTemplate() with M_ADD or M_INSERT), Aurora Imaging Library ignores them.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINING_WIDTH_NOMINAL		Sets the width with which to validate the template's automatically established nominal width. M_TRAINING_WIDTH_NOMINAL only applies when M_WIDTH_NOMINAL_MODE is set to M_AUTO_.... The width of the measured bead automatically established with the training image must respect the nominal width specified, otherwise the template will be invalid. To set scaling factors based on the nominal width, use M_TRAINING_WIDTH_SCALE_MAX and M_TRAINING_WIDTH_SCALE_MIN. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_NOMINAL has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the width, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_WIDTH_SCALE_MAX		Sets the scale factor by which to determine the upper limit (maximum permitted scale) of the bead width. Aurora Imaging Library applies the scale factor to the nominal training width (M_TRAINING_WIDTH_NOMINAL). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.2.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_SCALE_MAX has no effect.	c		e	f	g	a		i		j	k	l	m	n	b

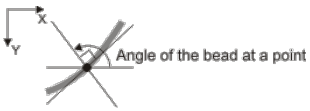
	Value > 0.0	Specifies the maximum scale factor. For example, a maximum scale factor of 1.2 indicates that the width of the measured bead can be up to 20% bigger than the nominal training width specified.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_WIDTH_SCALE_MIN		Sets the scale factor by which to determine the lower limit (minimum permitted scale) of the bead width. Aurora Imaging Library applies the scale factor to the nominal width (M_TRAINING_WIDTH_NOMINAL). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.8.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_SCALE_MIN has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the minimum scale factor. For example, a minimum scale factor of 0.8 indicates that the width of the measured bead can be up to 20% smaller than the nominal training width specified.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_NOMINAL		Sets the nominal width to use, when it is explicitly defined. M_WIDTH_NOMINAL only applies when M_WIDTH_NOMINAL_MODE is set to M_USER_DEFINED. The width set for M_WIDTH_NOMINAL applies to all vertices. To modify the width of one or more vertices, use MbeadTemplate() with M_SET_WIDTH_NOMINAL. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the nominal width, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_WIDTH_NOMINAL_MODE		Sets how to establish the nominal width of the bead template. If the bead is a stripe (MbeadTemplate() with M_BEAD_STRIPE), the width refers to the thickness between the stripe's outer edges. If the bead is an edge (M_BEAD_EDGE), the width refers to the thickness of the edge's intensity transition. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_CONTINUOUS	Specifies that the bead's nominal width is automatically established with MbeadTrain(), where the corresponding width at each vertex in the template can have its own unique value. In this case, you must use a training image to train the template (MbeadTrain()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_UNIFORM	Specifies that the bead's nominal width is automatically established with MbeadTrain(), where the corresponding width at all of the template's vertices must have the same value. The width corresponds to the average bead width. In this case, you must use a training image to train the template (MbeadTrain()).	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that you must explicitly set the bead's nominal width. To specify the nominal width of the bead, use M_WIDTH_NOMINAL. To specify the width of the bead corresponding to one or more vertices in the template, use MbeadTemplate() with M_SET_WIDTH_NOMINAL.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the templates' (one or all) corresponding measured bead for the verification phase (MbeadVerify()). In this case, you must set the LabelOrIndex parameter to a specific template (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()) or to all templates (M_ALL).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control measured beads for the verification phase																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_ACCEPTANCE		Sets the acceptance level for the score of the template's corresponding measured bead. The measured bead can only have a passing status result (M_STATUS) if its score is greater than or equal to the acceptance, regardless of any other setting. To retrieve the computed score, use MbeadGetResult() with M_SCORE. The score indicates, as a percentage, the amount of trained points in the template that have a corresponding measured point with a passing status, relative to the total number of trained points in the template. INQ														
	M_DEFAULT	Specifies the default value; the default value is 100.0%.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level, as a percentage. 100.0% indicates that, for every trained point in the template, there is a corresponding measured point that has a passing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_ACCURACY_MAX_DEVIATION		Sets the maximum angular deviation between the angle of the bead template at each of its trained points, and the angle of the corresponding measured bead at each of its measured points. Aurora Imaging Library considers the angle represented by the trained and measured points to be along the theoretical line that is perpendicular between the point and its corresponding bead. INQ 	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0°.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 180.0	Specifies the angular deviation, in degrees. If you specify 0° (no deviation), Aurora Imaging Library considers the angle of the measured bead at its measured points to be the same as the angle of the bead template at its corresponding trained points. Deviation values should typically be only a few degrees, which can allow you to manage minor fluctuations in bead width that can come from angular differences in certain target images. Larger deviation values can lead to invalid results and longer processing times.	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_WIDTH_MARGIN		Sets an additional width for the search box Aurora Imaging Library uses to verify the template's measured bead. Aurora Imaging Library bases the search box's width on the trained box's width, which you can inquire with M_TRAINED_BOX_WIDTH. Aurora Imaging Library adds the additional width (M_BOX_WIDTH_MARGIN) to the sides (margins) of the search box. Aurora Imaging Library applies the additional width as a percentage of the trained nominal width of the bead template, which you can inquire with M_TRAINED_WIDTH_NOMINAL. That is, *SearchBoxWidth* = *TrainedBoxWidth* + (*TrainedNominalWidth* x *AdditionalMarginPercentage*). For example, if the width of the trained box is 100 pixels, the trained nominal width of the bead is 60 pixels, and the additional width for the search box is 50%, the search box width is 130 pixels (100 + (60 x 0.5). In this case, the search box is 30 pixels wider (15 pixels on each if its sides) than the trained box. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 50.0%.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the additional width used for verification, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_WARNING_OFFSET		Sets an additional width to the search box in which a measured point can be found, but will have a failed status result (M_STATUS). When measured points fall outside the search box, but within this additional width, you can still retrieve results about them, such as their found position. If M_FAIL_WARNING_OFFSET is less than the search box width that Aurora Imaging Library uses for validating measured points (M_BOX_WIDTH_MARGIN), M_FAIL_WARNING_OFFSET has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the width, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_MAX_LENGTH		Sets the maximum allowable length of one single gap in the template's measured bead, for it to have a passing status result (M_STATUS). A gap refers to a section of the measured bead that corresponds to a single trained point, or to a set of consecutive trained points, that were expected but not found during verification. If any gap is longer than M_GAP_MAX_LENGTH, the template's measured bead will not have a passing status, regardless of any other setting. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_GAP_MAX_LENGTH has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum gap length, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS. A value of 0.0 indicates that no gaps are allowed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_TOLERANCE		Sets the percentage of the total allowable gap (all gaps together) in the template's measured bead, for it to have a passing status result (M_STATUS). A gap refers to a section of the measured bead that corresponds to a single trained point, or to a set of consecutive trained points, that were expected but not found during verification. If the total combined length of all gaps in the measured bead, relative to the full length of the corresponding bead template, is greater than M_GAP_TOLERANCE, the template's measured bead will not have a passing status, regardless of any other setting. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0%.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the total allowable gaps, as a percentage. A value of 0.0% indicates that no gaps are allowed.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTENSITY_DELTA_NEG		Sets the lowest pixel intensity that is considered valid for the measured bead, relative to the nominal intensity. That is, *LowestValidIntensity* = *NominalIntensity* - M_INTENSITY_DELTA_NEG. The nominal intensity can either be established by the training phase (M_INTENSITY_NOMINAL_MODE set to M_AUTO) or with an explicit value (M_INTENSITY_NOMINAL). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the lowest valid pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_DELTA_POS		Sets the highest pixel intensity that is considered valid for the measured bead, relative to the nominal intensity. That is, *HighestValidIntensity* = *NominalIntensity* + M_INTENSITY_DELTA_POS. The nominal intensity can either be established by the training phase (M_INTENSITY_NOMINAL_MODE set to M_AUTO) or with an explicit value (M_INTENSITY_NOMINAL). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the highest valid pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b

M_INTENSITY_NOMINAL		Sets the nominal pixel intensity of the template's measured bead. M_INTENSITY_NOMINAL only has an effect if you set M_INTENSITY_NOMINAL_MODE to M_USER_DEFINED. To specify a range of acceptable pixel-intensities for the measured bead, use M_INTENSITY_DELTA_NEG and M_INTENSITY_DELTA_POS. You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the nominal pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_MAX		Sets the maximum allowable distance between the position of the trained points in the template, and the position of the corresponding measured points in the target, for the measured bead to have a passing status result (M_STATUS). Measured points must always fall within the search box (for example, M_BOX_WIDTH_MARGIN). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_OFFSET_MAX has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the allowable offset, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_WIDTH_DELTA_NEG		Sets the valid delta negative tolerance between the width of the measured bead and the nominal width of the corresponding template, for the measured bead to have a passing status result (M_STATUS). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_WIDTH_DELTA_NEG has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the tolerance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_DELTA_POS		Sets the valid delta positive tolerance between the width of the measured bead and the nominal width of the corresponding template, for the measured bead to have a passing status result (M_STATUS). You can only use this value with stripe-beads. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_WIDTH_DELTA_POS has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the tolerance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control how Aurora Imaging Library performs the edge extraction of the bead from the training (MbeadTrain()) and target (MbeadVerify()) image. In this case, you must set the LabelOrIndex parameter to a specific template (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()) or to all templates (M_ALL).

Aurora Imaging Library extracts beads from images using processes based on the Aurora Imaging Library

Measurement module. Such processes use a one-dimensional analysis of differences in pixel intensities. For more information about this type of edge extraction, see the Search algorithm section of Chapter 19: Measurement.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control how Aurora Imaging Library extracts beads from an image for the training and verification phases

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FOREGROUND_VALUE	Sets whether the foreground that Aurora Imaging Library uses when performing the edge extraction of the bead from an image should be darker or lighter than the background. Only beads with the specified foreground will be trained or verified, depending on which function you are calling. INQ															
M_DEFAULT	Same as M_FOREGROUND_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_BLACK	Specifies that the foreground is darker than the background. For example, a black bead on a white surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_FOREGROUND_WHITE	Specifies that the foreground is lighter than the background. For example, a white bead on a black surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTHNESS	Sets the degree of noise reduction that Aurora Imaging Library uses when performing the edge extraction of the bead from an image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the smoothness value. A setting of 0.0 indicates almost no noise reduction effect, while a setting of 100.0 indicates a very strong noise reduction effect.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE	Sets how to establish the threshold that Aurora Imaging Library must use when performing the edge extraction of the bead from an image. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the threshold is explicitly defined with M_THRESHOLD_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE		Sets the value beneath which a grayscale variation within the image is not considered an edge of the bead. This value only applies if M_THRESHOLD_MODE is set to M_USER_DEFINED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the threshold value. To consider all grayscale variations as possible edges of the bead, irrespective of their grayscale value, set M_THRESHOLD_VALUE to 0.0. However, in this case, due to the increased number of edges, processing time might increase, and false positives might occur.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to associate a camera calibration context with the training image. In this case, you must set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To associate a camera calibration context with a training image																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_ASSOCIATED_CALIBRATION		Associates the specified camera calibration context with the training image specified in MbeadTrain(), if required. Aurora Imaging Library requires a calibrated training image if your training settings are specified in world units (M_TEMPLATE_INPUT_UNITS and/or M_TRAINING_BOX_INPUT_UNITS set to M_WORLD). If you associate a camera calibration, and you pass a calibrated training image to MbeadTrain(), Aurora Imaging Library ignores the associated camera calibration (M_ASSOCIATED_CALIBRATION), and uses the image's camera calibration instead. This value is for the training phase (MbeadTrain()). If you modify this value, you must retrain the context before the next verification. INQ															
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Removes the association between the training image and the camera calibration context.	c		e	f	g	a		i		j	k	l	m	n	b
	Calibration context identifier	Specifies the camera calibration context to associate with the training image.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control operations with MbeadGetNeighbors(). In this case, you must set the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To control operations with MbeadGetNeighbors()																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_CLOSEST_POINT_MAX_DISTANCE		Sets the maximum distance for the M_CLOSEST_... operations in MbeadGetNeighbors(). INQ															
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the maximum distance is infinite.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies the maximum distance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b	

Type-specific versions of the function when using a C compiler under 64-bit

```
void MbeadControlInt64 (AIL_ID ContextBeadId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

ContextBeadId

See ContextBeadId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MbeadControlDouble (AIL_ID ContextBeadId, AIL_INT LabelOrIndex, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextBeadId

See ContextBeadId of the main function for a description.

LabelOrIndex

See LabelOrIndex of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadDraw

Synopsis

Draw specific features of a bead context or of a bead result.

Syntax

```
void MbeadDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResultBeadId,  //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 DrawOperation,       //in
    AIL_INT64 DrawOption,          //in
    AIL_INT LabelOrIndex,          //in
    AIL_INT Index,                 //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of a selected bead context or bead result, in the specified destination image buffer or 2D graphics list. Drawings are typically performed using either the template's vertices or trained points (when specifying a bead context), or using the trained points' corresponding measured point (when specifying a bead result). Drawing operations for trained points are only available after calling MbeadTrain(), while drawing operations for bead results are only available after calling MbeadVerify().

You can draw results and settings, obtained relative to an offset, at the top-left corner of the destination image, using MgraControl() with M_DRAW_OFFSET_X and M_DRAW_OFFSET_Y and zoom them using MgraControl() with M_DRAW_ZOOM_X and M_DRAW_ZOOM_Y. For example, you can draw a zoomed section of the bead found in the target image at the top-left corner of the destination image. For more information, see the Drawing graphics with offset and zoom subsection of the Drawing graphics section of Chapter 26: Generating graphics.

All operations are available for drawing in either the specified destination image buffer or 2D graphics list, unless otherwise specified.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultBeadId

Specifies the identifier of the bead context or bead result buffer that this function will use to perform the drawing operation. The context or result buffer must have been previously allocated on the system using MbeadAlloc() or MbeadAllocResult(), respectively.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When calling **MbeadDraw**() with a bead context (and using a calibrated training image) or with a bead result (and using a calibrated target image), the destination drawing buffer need not be calibrated to the same world. Aurora Imaging Library draws according to the calibration context associated to either the training or verification image.

DrawOperation

Specifies the drawing operation to perform.

To perform drawing operations for a bead context or a bead result, this parameter can be set to one of the following values. These drawing operations are performed according to the points specified with the DrawOption parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Drawing operations for a context or result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_INDEX	Draws the index of the templates containing the points that fulfill the conditions of the DrawOption parameter. For example, if you set the DrawOption parameter to M_TRAINED, the index of all the successfully trained templates will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b

M_DRAW_LABEL	Draws the label of the templates containing the points that fulfill the conditions of the DrawOption parameter. For example, if you set the DrawOption parameter to M_FAIL, the label of all the templates with at least one failed point will be drawn.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION	Draws a cross at the position of the point.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION_INDEX	Draws the index value of the point. If you specify M_USER, you can only perform this drawing operation for templates with a path that follows a polyline (MbeadControl() with M_TRAINING_PATH), otherwise you will get an error.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION_POLYLINE	Draws the polyline that connects the points. If the template's path follows a circle, Aurora Imaging Library draws an actual circle (smooth curve).	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_SEARCH_BOX	Draws the search box of the point.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_WIDTH	Draws an H-type line (|-|) indicating the width of the bead at that point. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b

To draw the training image, this parameter must be set to the following value. To draw the training image, you must set the ContextOrResultBeadId parameter to a bead context, the DstImageBufOrListGraId parameter to a destination image, the DrawOption parameter to M_DEFAULT, the LabelOrIndex parameter to M_GENERAL, and the Index parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Drawing the training image																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_TRAINING_IMAGE	Draws the training image. This operation cannot draw in a 2D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b

DrawOption

Specifies the type of points that this function will use to perform the drawing operation. If this information is not required for the specified drawing operation, set this parameter to M_DEFAULT.

When performing a drawing operation for a bead context or bead result, this parameter can be set to the

following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing a drawing operation with a context or a result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. For a bead context, the default value is the same as M_TRAINED. For a bead result, the default value is the same as M_ALL. For an M_DRAW_TRAINING_IMAGE operation, the default indicates that this parameter's information is not required.	c		e	f	g	a		i		j	k	l	m	n	b

When performing a drawing operation for a bead context, this parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing a drawing operation with a context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TRAINED	Specifies that Aurora Imaging Library performs the drawing operation using the trained points indicated by both M_TRAINED_FAIL and M_TRAINED_PASS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_FAIL	Specifies that Aurora Imaging Library performs the drawing operation using all the trained points that were expected but not found, during the training phase (MbeadTrain()). In this case, the drawing operation uses the expected position of the trained point.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_PASS	Specifies that Aurora Imaging Library performs the drawing operation using all the trained points that were established (found), during the training phase MbeadTrain().	c		e	f	g	a		i		j	k	l	m	n	b

M_USER	Specifies that Aurora Imaging Library performs the drawing operation using the vertices of the template. If the template's path follows a polyline (MbeadControl() with M_TRAINING_PATH), M_USER refers to the specified polyline points (MbeadTemplate()). If the template's path follows a circle or segment, M_USER refers to the minimum critical points that Aurora Imaging Library internally calculates based on the specified settings of the path. For a path that follows a segment, Aurora Imaging Library calculates three critical points, according to the specified start and end of the segment (MbeadControl() with M_TEMPLATE_SEGMENT_...). For a path that follows a circle, Aurora Imaging Library calculates five critical points, according to the specified center and radius of the circle (M_TEMPLATE_CIRCLE_...). You cannot specify M_USER for an M_DRAW_WIDTH operation; Aurora Imaging Library considers width a trained aspect of the bead (as is the case, for example, when you are using a training image to establish the width).	c		e	f	g	a		i		j	k	l	m	n	b

When performing a drawing operation for a bead result, this parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing a drawing operation with a result																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies that the drawing operation will be performed using all measured points, regardless of whether they have passed (M_PASS) or failed (M_FAIL). For measured points that were not found, the drawing operation uses their expected position.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that the drawing operation will be performed using all measured points that were expected but did not pass verification. In this case, the drawing operation uses the expected position of the measured point.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_GAP_MAX	Specifies that the drawing operation will be performed using all measured points that have failed verification, and whose expected position falls within the maximum gap in the bead, as specified using MbeadControl() with M_GAP_MAX_LENGTH.	c		e	f	g	a		i		j	k	l	m	n	b

M_FAIL_INTENSITY_MAX	Specifies that the drawing operation will be performed using all measured points that have failed verification because their intensity is greater than the highest intensity allowed, as specified using: (MbeadControl() with M_INTENSITY_NOMINAL) + (MbeadControl() with M_INTENSITY_DELTA_POS). This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_INTENSITY_MIN	Specifies that the drawing operation will be performed using all measured points that have failed verification because their intensity is less than the lowest intensity allowed, as specified using: (MbeadControl() with M_INTENSITY_NOMINAL) - (MbeadControl() with M_INTENSITY_DELTA_NEG). This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_NOT_FOUND	Specifies that the drawing operation will be performed using all measured points that have failed verification because they were not found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_OFFSET	Specifies that the drawing operation will be performed using all measured points that have failed verification because their position has surpassed the offset criterion, as specified using MbeadControl() with M_OFFSET_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_WIDTH_MAX	Specifies that the drawing operation will be performed using all measured points that have failed verification because the width of the measured bead at their position is greater than the maximum width criterion, as specified using: (MbeadControl() with M_TRAINING_WIDTH_NOMINAL) + (MbeadControl() with M_WIDTH_DELTA_POS). This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_WIDTH_MIN	Specifies that the drawing operation will be performed using all measured points that have failed verification because the width of the measured bead at their position is less than the minimum width criterion, as specified using: (MbeadControl() with M_TRAINING_WIDTH_NOMINAL) - (MbeadControl() with M_WIDTH_DELTA_NEG). This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b

M_PASS	Specifies that the drawing operation will be performed using all the measured points that have passed verification.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies the templates (one or all) that this function will use to perform the drawing operation, or specifies that you are performing a general type of drawing operation. The specified template is either in a bead context or result buffer, depending on the ContextOrResultBeadId parameter setting. All LabelOrIndex parameter values can be used for any draw operation, unless otherwise specified.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the template																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of a template for which to perform the drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index. The index is from 0 (inclusive) to the number of templates minus 1.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of a template, for which to perform the drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label. The label must be greater than 0.															
M_ALL	Specifies to perform the drawing operation for all templates.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies that you are performing a general type of drawing operation. You can only use M_GENERAL with M_DRAW_TRAINING_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies the set of template points (one or all) for which to perform the drawing operation, or specifies that you are performing a general type of drawing operation. All Index parameter values can be used for any draw operation (except M_DRAW_TRAINING_IMAGE).

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of the trained point																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies to perform the drawing operation for all points in the specified templates.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies that you are performing a general type of drawing operation. In this case, the entire template is either drawn or not, depending on whether it contains one or more points that meet the conditions of the drawing operation. You must specify M_GENERAL when using M_DRAW_TRAINING_IMAGE.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the point in the specified templates for which to perform the drawing operation. The index is from 0 (inclusive) to the total number of points in the template minus 1. To get the number of trained points in the template, use MbeadGetResult() with M_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether Aurora Imaging Library performs drawings with fail-type points differently than drawings with other types of points.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether Aurora Imaging Library performs drawings with fail-type points differently																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that Aurora Imaging Library does not perform drawings with fail-type points differently.	c		e	f	g	a		i		j	k	l	m	n	b

M_ENHANCED	Specifies that Aurora Imaging Library performs drawings with fail-type points differently than drawings with other types of points. That is, when possible, drawing operations (M_DRAW_...) with M_..._FAIL or M_FAIL_... points, such as M_TRAINED_FAIL and M_FAIL_NOT_FOUND, will be visually distinguishable from drawing operations with other types of points, such as M_USER, M_TRAINED_PASS, and M_PASS. This can be useful when drawing both fail-type and pass-type points, either by calling **MbeadDraw**() multiple times, or by performing the drawing operation on all points (M_TRAINED and M_ALL). For example, if you specify M_DRAW_POSITION_INDEX and M_ALL, the index of each trained point is drawn, if its associated measured point has been found (pass); for measured points that were expected but not found (fail), the index of their associated trained point is drawn with a horizontal line above it. You cannot use M_ENHANCED when drawing the training image (M_DRAW_TRAINING_IMAGE).	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadFree

Synopsis

Free a bead context or a bead result buffer.

Syntax

```
void MbeadFree(
    AIL_ID ContextOrResultBeadId //in
)
```

Description

This function deletes the specified bead context or result buffer identifier, and releases any memory associated with it.

All bead contexts and all bead result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultBeadId

Specifies the identifier of the bead context or bead result buffer to free. These must have been successfully allocated (with MbeadAlloc() or MbeadAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadGetNeighbors

Synopsis

Get the template and/or vertex closest to a source point.

Syntax

```
void MbeadGetNeighbors(
    AIL_ID ContextBeadId,      //in
    AIL_INT LabelOrIndex,      //in
    AIL_DOUBLE PositionX,      //in
    AIL_DOUBLE PositionY,      //in
    AIL_INT *TemplateLabelPtr, //out
    AIL_INT *PointIndexPtr,    //out
    AIL_INT64 ControlFlag      //in
)
```

Description

This function retrieves the label of the template, and the vertex within that template, that is closest to the specified position. This function can also retrieve the vertex within the specified template that is closest to the specified position. You can only use this function with templates whose path follows a polyline (MbeadControl() with M_TRAINING_PATH set to either M_POLYLINE_SEED or M_POLYLINE).

To set a maximum distance for which Aurora Imaging Library considers a template or vertex to be the closest, use MbeadControl() with M_CLOSEST_POINT_MAX_DISTANCE. By default, there is no maximum distance.

Parameters

ContextBeadId

Specifies the identifier of the bead context. The bead context must have been previously allocated on the required system using MbeadAlloc().

LabelOrIndex

Specifies the bead template in which the closest vertex is expected to be located.

This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a bead template																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies no label or index value. When using M_CLOSEST_TEMPLATE, LabelOrIndex must be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of the template.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index. The index must be greater than or equal to 0.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of the template.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label. The label must be greater than 0.															

PositionX

Specifies the source point's X-position.

PositionY

Specifies the source point's Y-position.

TemplateLabelPtr

Specifies the address of the variable in which to write the label of the closest template.

When using M_CLOSEST_POINT, TemplateLabelPtr must be set to **M_NULL**.

If there is no template (in the case of M_CLOSEST_TEMPLATE) sufficiently close to the source point, **M_INVALID** will be returned.

PointIndexPtr

Specifies the address of the variable in which to write the index of the closest vertex.

If there is no template (in the case of M_CLOSEST_TEMPLATE) or vertex (in the case of M_CLOSEST_POINT) sufficiently close to the source point, **M_INVALID** will be returned.

ControlFlag

Specifies whether to retrieve the closest template and/or the closest vertex.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to receive the closest template or the closest vertex																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_CLOSEST_TEMPLATE.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLOSEST_POINT	Specifies to return the vertex, within the specified template, that is closest to the specified source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLOSEST_TEMPLATE	Specifies to return the template, and the vertex within that template, that are closest to the specified source point.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadGetResult

Synopsis

Get results from a bead result buffer.

Syntax

```
void MbeadGetResult(
    AIL_ID ResultBeadId,  //in
    AIL_INT LabelOrIndex, //in
    AIL_INT ResultIndex,  //in
    AIL_INT64 ResultType, //in
    void *UserVarPtr      //out
)
```

Description

This function retrieves results of the specified type from a bead result buffer. Results are only available after calling MbeadVerify().

Results for the specified bead template or trained point are based on how they compare with their corresponding measured bead or measured point in the verification's target image. The returned results are ordered by the label value of their template.

If your target image was associated with a camera calibration context, positional and dimensional results are returned with respect to the relative coordinate system of the target image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

Parameters

ResultBeadId

Specifies the identifier of the bead result buffer from which to get results.

LabelOrIndex

Specifies the label or index of the template for which to get results, or specifies that you are getting general results related to all templates. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying where you are getting results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of the template for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	Index Specifies the index. Valid indices range from 0 (the first template) to the number of defined templates minus 1.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of the template for which to get results.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label. The label must be greater than 0.															
M_GENERAL	Specifies to get general results related to all templates.	c		e	f	g	a		i		j	k	l	m	n	b

ResultIndex

Specifies the trained points (one or all) for which to get results or specifies that you are getting general results. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the trained points for which to get results or for specifying general results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to get results for each trained point in the template. In this case, the LabelOrIndex parameter must be set to M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to get general results. To get general results related to all templates, the LabelOrIndex parameter must be set to M_GENERAL. To get general results related to a specific template, the LabelOrIndex parameter must be set to M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the index of the trained point for which to get results. Valid indices range from 0 (the first point) to the total number of trained points in the template minus 1. To get the number of trained points in the template, use **MbeadGetResult**() with M_NUMBER. When specifying the index of the trained point, the LabelOrIndex parameter must be set to M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the result to get.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **UserVarPtr** parameter.

- For retrieving the status of results
- For retrieving the number of templates or the number of trained points in a template
- For retrieving results related to a template (measured bead)
- For retrieving results related to trained points (measured points)
- For retrieving results related to a template (measured bead) or to trained points (measured points)

To get the status of results, the ResultType parameter can be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the status of results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_STATUS +		Retrieves whether results have passed or failed verification. To retrieve the status of results related to all templates, you must set the LabelOrIndex and the ResultIndex parameter to M_GENERAL. In this case, you will get a passing status only if all status results (M_STATUS_...) of all templates have passed verification. To retrieve the status of results related to a specific template, you must set the LabelOrIndex parameter to the label or index of the template and the ResultIndex parameter to M_GENERAL. In this case, you will get a passing status only if the template's corresponding measured bead has passed all of the following: M_STATUS_GAP_TOLERANCE, M_STATUS_GAP_MAX, and M_STATUS_SCORE. To retrieve the status of results related to one or all trained points, you must set the LabelOrIndex parameter to the label or index of a template and the ResultIndex parameter to a specific value or M_ALL. In this case, you will get a passing status only if the trained point's corresponding measured point has passed all of the following: M_STATUS_SEARCH, M_STATUS_FOUND, M_STATUS_OFFSET, M_STATUS_WIDTH_MAX, M_STATUS_WIDTH_MIN, and M_STATUS_GAP_MAX.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAIL	Specifies that the results have not passed verification.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PASS	Specifies that the results have passed verification.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve the number of templates or the number of trained points in a template, the ResultType parameter can be set to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the number of templates or the number of trained points in a template																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_NUMBER +	Retrieves the number of templates or the number of trained points in a template. To retrieve the number of templates, you must set the LabelOrIndex and the ResultIndex parameters to M_GENERAL. To retrieve the number of trained points in a template, you must set the LabelOrIndex parameter to the index or label of the template and the ResultIndex parameter to M_GENERAL.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results related to a template's corresponding measured bead, the ResultType parameter should be set to one of the following values. In this case, you must set the LabelOrIndex parameter to the index or label of the template for which to get results (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()) and the ResultIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results related to a template (measured bead)																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_CLOSURE +		Retrieves whether the measured bead is closed.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the measured bead is not closed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the measured bead is closed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_COVERAGE +		Retrieves the percentage of the total gap in the measured bead, relative to the length of the expected measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_MAX_LENGTH +		Retrieves the length of the longest uninterrupted chain of trained points that were not found in the bead.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_INTENSITY_MAX +	Retrieves the maximum pixel intensity of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MAX_INDEX +	Retrieves the index of the trained point that corresponds to the maximum intensity of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that Aurora Imaging Library could not establish a maximum pixel intensity. This can occur if, for example, no bead was found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the trained point that corresponds to the maximum intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MIN +	Retrieves the minimum pixel intensity of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_MIN_INDEX +	Retrieves the index of the trained point that corresponds to the minimum intensity of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that Aurora Imaging Library could not establish a minimum pixel intensity. This can occur if, for example, no bead was found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the trained point that corresponds to the minimum intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_LABEL_VALUE +	Retrieves the label of the template corresponding to the index specified with M_TEMPLATE_INDEX().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_NUMBER_FOUND +	Retrieves the number of trained points that have a corresponding measured point in the bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_MAX +	Retrieves the maximum linear distance between the position of a trained point and the position of its corresponding measured point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_MAX_INDEX +	Retrieves the index of the trained point that represents the maximum offset of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that Aurora Imaging Library could not establish a maximum offset. This can occur if, for example, no bead was found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the index of the trained point that corresponds to the maximum offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +	Retrieves the percentage of trained points that have a corresponding measured point with a passing status, relative to the total number of trained points in the template.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_GAP_TOLERANCE +	Retrieves whether the M_GAP_COVERAGE result has passed or failed the gap tolerance criterion, as specified using MbeadControl() with M_GAP_TOLERANCE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that the result is greater than the gap tolerance criterion.	c		e	f	g	a		i		j	k	l	m	n	b

M_PASS	Specifies that the result is less than or equal to the gap tolerance criterion.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_SCORE +	Retrieves whether the M_SCORE result has passed or failed the acceptance criterion, as specified using MbeadControl() with M_ACCEPTANCE.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that the result is greater than the acceptance criterion.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies that the result is less than or equal to the acceptance criterion.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_AVERAGE +	Retrieves the average width of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_MAX +	Retrieves the maximum width of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_MAX_INDEX +	Retrieves the index of the trained point that corresponds to the maximum width of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that Aurora Imaging Library could not establish a maximum width. This can occur if, for example, no bead was found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the trained point that corresponds to the maximum width.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_MIN +	Retrieves the minimum width of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_WIDTH_MIN_INDEX +	Retrieves the index of the trained point that corresponds to the minimum width of the measured bead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that Aurora Imaging Library could not establish a minimum width. This can occur if, for example, no bead was found in the target image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the index of the trained point that corresponds to the minimum width.	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results related to a trained point's (one or all) corresponding measured point, the ResultType parameter should be set to one of the following values. In this case, you must set the LabelOrIndex parameter to the index or label of the template for which to get results (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()) and the ResultIndex parameter to a specific value or M_ALL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results related to trained points (measured points)																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ANGLE +	Retrieves the angle of the measured bead at the position of the measured point.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_END_POS_X +	Retrieves the X-coordinate of the right edge of the bead, at a point that is collinear with the measured point and a corresponding point on the left edge. Note that left and right are established from the bead's direction, which follows the sequence of measured points in increasing order. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_END_POS_Y +	Retrieves the Y-coordinate of the right edge of the bead, at a point that is collinear with the measured point and a corresponding point on the left edge. Note that left and right are established from the bead's direction, which follows the sequence of measured points in increasing order. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY +	Retrieves the intensity of the measured bead at the position of the measured point. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET +	Retrieves the linear distance between the trained point and its corresponding measured point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Retrieves the X-coordinate of the measured point. The position is taken at the center of the bead width, at that point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +	Retrieves the Y-coordinate of the measured point. The position is taken at the center of the bead width, at that point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_START_POS_X +	Retrieves the X-coordinate of the left edge of the bead, at a point that is collinear with the measured point and a corresponding point on the right edge. Note that left and right are established from the bead's direction, which follows the sequence of measured points in increasing order. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_START_POS_Y +	Retrieves the Y-coordinate of the left edge of the bead, at a point that is collinear with the measured point and a corresponding point on the right edge. Note that left and right are established from the bead's direction, which follows the sequence of measured points in increasing order. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_SEARCH +	Retrieves whether Aurora Imaging Library considers it possible to establish the measured point, given the current settings. For example, if the search box corresponding to a measured point falls outside the target image, Aurora Imaging Library returns M_FAIL since you can never establish a measured point that is beyond the boundaries of the image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies that Aurora Imaging Library considers it impossible to establish the measured point.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies that Aurora Imaging Library considers it possible to establish the measured point.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_INDEX +	Retrieves the index of the trained point that corresponds to the measured point. This can be useful to get the indices of all trained points (M_ALL).	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_POSITION_X +	Retrieves the X-position of the trained point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_POSITION_Y +	Retrieves the Y-position of the trained point.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_VALUE +	Retrieves the width of the measured bead at the position of the measured point. This value is only supported for stripe-beads.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

To retrieve results related to a template's corresponding measured bead, or to a trained point's (one or all) corresponding measured point, the ResultType parameter can be set to one of the following values. In this case, you must set the LabelOrIndex parameter to the index or label of the template for which to get results (M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL()). When retrieving results related to a template, you must set the ResultIndex parameter to M_GENERAL. When retrieving results related to the trained points, you must set the ResultIndex parameter to a specific value or M_ALL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving results related to a template (measured bead) or to trained points (measured points)																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_STATUS_FOUND +	Retrieves the found status of the related template or trained point. If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all trained points have a corresponding measured point (all were found). Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether the corresponding measured point was found. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_GAP_MAX +	Retrieves whether the M_GAP_MAX_LENGTH result has passed or failed the maximum gap criterion, as specified using MbeadControl() with M_GAP_MAX_LENGTH. If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if the longest uninterrupted chain of trained points that were not found in the measured bead is less than M_GAP_MAX_LENGTH. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its associated measured point was not found, and is part of a gap that has failed the maximum gap criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b

M_STATUS_INTENSITY_MAX +		Retrieves whether the M_INTENSITY result has passed or failed the criterion for the highest pixel intensity allowed, as specified using MbeadControl() with *NominalIntensity* + M_INTENSITY_DELTA_POS. This value is only supported for stripe-beads. The nominal intensity can either be established by the training phase (M_AUTO) or with an explicit value (M_INTENSITY_NOMINAL). If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all measured points have an intensity that is lower than the highest intensity allowed. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its corresponding measured point has failed or passed the maximum color criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b

M_STATUS_INTENSITY_MIN +	Retrieves whether the M_INTENSITY result has passed or failed the criterion for the lowest intensity allowed, as specified using MbeadControl() with *NominalIntensity* - M_INTENSITY_DELTA_NEG. This value is only supported for stripe-beads. The nominal intensity can either be established by the training phase (M_AUTO) or with an explicit value (M_INTENSITY_NOMINAL). If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all measured points have an intensity that is higher than the lowest intensity allowed. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its measured point has failed or passed the minimum color criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_OFFSET +	Retrieves whether the M_OFFSET result has passed or failed the maximum offset criterion, as specified using MbeadControl() with M_OFFSET_MAX. If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all measured points pass the maximum offset criterion. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its measured point has failed or passed the maximum offset criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b

M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_WIDTH_MAX +	Retrieves whether the M_WIDTH_VALUE result has passed or failed the maximum width criterion, as specified using MbeadControl() with M_TRAINING_WIDTH_NOMINAL + M_WIDTH_DELTA_POS. If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all measured points pass the maximum width criterion. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its measured point has failed or passed the maximum width criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS_WIDTH_MIN +	Retrieves whether the M_WIDTH_VALUE result has passed or failed the minimum width criterion, as specified using MbeadControl() with M_TRAINING_WIDTH_NOMINAL - M_WIDTH_DELTA_NEG. If you set the ResultIndex parameter to M_GENERAL, M_PASS is returned if all measured points pass the minimum width criterion. Otherwise, M_FAIL is returned. If you set the ResultIndex parameter to M_ALL, M_FAIL or M_PASS is returned for each trained point, depending on whether its measured point has failed or passed the minimum width criterion. If you set the ResultIndex parameter to a specific trained point, only the status of that point is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL	Specifies a failing status.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PASS	Specifies a passing status.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving the status of results
- For retrieving the number of templates or the number of trained points in a template
- For retrieving results related to a template (measured bead)
- For retrieving results related to trained points (measured points)
- For retrieving results related to a template (measured bead) or to trained points (measured points)

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to specify whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_AVAILABLE		Retrieves whether the requested result type is available for retrieval.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving the status of results
- For retrieving results related to trained points (measured points)
- For retrieving results related to a template (measured bead) or to trained points (measured points)

the values listed in all the parameter association tables except **For retrieving the number of templates or the number of trained points in a template**, **For retrieving results related to a template (measured bead)**.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values

ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For retrieving the status of results
- For retrieving the number of templates or the number of trained points in a template
- For retrieving results related to a template (measured bead)
- For retrieving results related to trained points (measured points)
- For retrieving results related to a template (measured bead) or to trained points (measured points)

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadInquire

Synopsis

Inquire information about a bead context, template, or result.

Syntax

```
AIL_INT MbeadInquire(
    AIL_ID ContextOrResultBeadId, //in
    AIL_INT LabelOrIndex,         //in
    AIL_INT64 InquireType,        //in
    void *UserVarPtr              //out
)
```

Description

This function inquires information about a specified bead context, a template contained within the context, or a bead result buffer. You can also use this function to inquire about operations performed with MbeadGetNeighbors(). Values are returned in the unit in which they were set (that is, either pixel or world units), using MbeadControl() and M_..._INPUT_UNITS.

If the inquired setting is set to M_DEFAULT (for example, in MbeadControl()), **MbeadInquire**() will return M_DEFAULT. To inquire the actual default value, add M_DEFAULT to the InquireType parameter.

Parameters

ContextOrResultBeadId

Specifies the identifier of the bead context or the identifier of the bead result buffer about which to inquire. Both the bead context and result buffer must have been previously allocated on the required system using MbeadAlloc() or MbeadAllocResult(), respectively.

LabelOrIndex

Specifies the bead template about which to inquire, or specifies that you are inquiring about a global setting of the bead context or result. Unless otherwise specified, the LabelOrIndex parameter values require that the ContextOrResultBeadId parameter specifies the identifier of a bead context.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to inquire about																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default. If ContextOrResultBeadId is set to a bead context, the default is the same as M_CONTEXT. If ContextOrResultBeadId is set to a bead result, the default is the same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of an existing template about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index. The index must be greater than or equal to 0.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of an existing template about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label. The label must be greater than 0.															
M_CONTEXT	Inquires information about a global setting of a bead context.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Inquires information about a setting of a bead result buffer. When using M_GENERAL, ContextOrResultBeadId must specify a bead result buffer.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MbeadInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- To inquire about a bead template setting for the training phase
- To inquire about a bead template setting for the training and verification phases
- To inquire about a trained setting of a bead template
- To inquire about a bead template setting for the verification phase
- To inquire about a global setting of a bead template
- To inquire about a global setting of a bead context
- To inquire about a global setting of a bead context or bead result buffer
- To inquire about settings for operations with MbeadGetNeighbors()

To inquire about a bead template setting for the training phase, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a bead template setting for the training phase																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_INDIVIDUAL_WIDTH_NOMINAL +		Inquires the specified width of the template at the position of each of its vertices. This value applies only to templates whose path follows a polyline. SET															
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_NOMINAL_MODE +		Inquires how to establish the nominal pixel intensity of the template. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO	Specifies to establish the nominal pixel intensity from the training image.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that M_INTENSITY_NOMINAL_MODE has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that you will explicitly set the pixel intensity, using M_INTENSITY_NOMINAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +		Inquires the X-coordinates of the vertices in a template whose path follows a polyline. You can add, delete, or modify a template's vertices using MbeadTemplate(). SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinates of the template's vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +		Inquires the Y-coordinates of the vertices in a template whose path follows a polyline. You can add, delete, or modify a template's vertices using MbeadTemplate(). SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinates of the template's vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE +		Inquires the number of vertices in a template whose path follows a polyline. You can add, delete, or modify a template's vertices using MbeadTemplate(). SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of points.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_CENTER_X +		Inquires the X-coordinate of the center of a template whose path follows a circle. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the center's X-coordinate, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_CENTER_Y +		Inquires the Y-coordinate of the center of a template whose path follows a circle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the center's Y-coordinate, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_CIRCLE_RADIUS +		Inquires the radius of a template whose path follows a circle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the radius, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_INPUT_UNITS +		Inquires the units with which to interpret the M_CLOSEST_POINT_MAX_DISTANCE, M_FAIL_WARNING_OFFSET, M_GAP_MAX_LENGTH, M_OFFSET_MAX, M_TEMPLATE_CIRCLE_CENTER_X, M_TEMPLATE_CIRCLE_CENTER_Y, M_TEMPLATE_CIRCLE_RADIUS, M_TEMPLATE_SEGMENT_END_X, M_TEMPLATE_SEGMENT_END_Y, M_TEMPLATE_SEGMENT_START_X, M_TEMPLATE_SEGMENT_START_Y, M_WIDTH_DELTA_NEG, M_WIDTH_DELTA_POS, and M_WIDTH_NOMINAL inquire types, as well as the units with which to interpret the template's position and dimension values specified using MbeadTemplate(). This essentially inquires the input coordinate system used. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_END_X +		Inquires the X-coordinate of the end of a template whose path follows a segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the end of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEMPLATE_SEGMENT_END_Y +		Inquires the Y-coordinate of the end of a template whose path follows a segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate of the end of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_START_X +		Inquires the X-coordinate of the start of a template whose path follows a segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate of the start of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEMPLATE_SEGMENT_START_Y +		Inquires the Y-coordinate of the start of a template whose path follows a segment. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate of the start of the segment, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_BOX_HEIGHT +		Inquires the height of the search boxes. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the height, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_BOX_INPUT_UNITS +		Inquires the units with which to interpret the M_TRAINING_BOX_HEIGHT, M_TRAINING_BOX_WIDTH, M_TRAINING_BOX_SPACING, and M_TRAINING_WIDTH_NOMINAL inquire types. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_PIXEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies to interpret the values in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies to interpret the values in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_BOX_SPACING +		Inquires the distance between the search boxes. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_BOX_SPACING has no effect. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the spacing, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_BOX_WIDTH +		Inquires the width of the search boxes. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 200.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the width, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_PATH +		Inquires how Aurora Imaging Library establishes the path of the bead template. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_POLYLINE_SEED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CIRCLE	Specifies that the path of the bead template follows a fixed circle defined with M_TEMPLATE_CIRCLE_CENTER_X, M_TEMPLATE_CIRCLE_CENTER_Y, and M_TEMPLATE_CIRCLE_RADIUS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POLYLINE	Specifies that the path of the bead template follows a fixed polyline. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_POLYLINE_SEED	Specifies that the path of the bead template follows a polyline that is refined by the training image set with MbeadTrain(). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SEGMENT	Specifies that the path of the bead template follows a fixed segment defined with M_TEMPLATE_SEGMENT_START_X, M_TEMPLATE_SEGMENT_START_Y, M_TEMPLATE_SEGMENT_END_X, and M_TEMPLATE_SEGMENT_END_Y. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_WIDTH_NOMINAL +		Inquires the width with which to validate the template's automatically established nominal width. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_NOMINAL has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the width, relative to the input coordinate system specified using M_TRAINING_BOX_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_WIDTH_SCALE_MAX +		Inquires the scale factor by which to determine the upper limit (maximum permitted scale) of the bead width. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.2.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_SCALE_MAX has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum scale factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_WIDTH_SCALE_MIN +		Inquires the scale factor by which to determine the lower limit (minimum permitted scale) of the bead width. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.8.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_TRAINING_WIDTH_SCALE_MIN has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the minimum scale factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_WIDTH_NOMINAL +	Inquires the nominal width to use, when it is explicitly defined. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the nominal width, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_NOMINAL_MODE +	Inquires how to establish the nominal width of the bead template. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_AUTO_UNIFORM.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_CONTINUOUS	Specifies that the bead's nominal width is automatically established with MbeadTrain(), where the corresponding width at each vertex in the template can have its own unique value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO_UNIFORM	Specifies that the bead's nominal width is automatically established with MbeadTrain(), where the corresponding width at all of the template's vertices must have the same value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_USER_DEFINED	Specifies that you must explicitly set the bead's nominal width. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a bead template setting for the training and verification phases, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a bead template setting for the training and verification phases																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_FOREGROUND_VALUE +		Inquires whether the foreground of the measured bead should be darker or lighter than the background. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FOREGROUND_WHITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_BLACK	Specifies that the foreground is darker than the background. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FOREGROUND_WHITE	Specifies that the foreground is lighter than the background. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SMOOTHNESS +		Inquires the degree of noise reduction that Aurora Imaging Library uses when performing the edge extraction of the bead from an image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the smoothness value. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_MODE +		Inquires how to establish the threshold that Aurora Imaging Library must use when performing the edge extraction of the bead from an image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_USER_DEFINED	Specifies that the threshold is explicitly defined with M_THRESHOLD_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_THRESHOLD_VALUE +		Inquires the value beneath which a grayscale variation within the image is not considered an edge of the bead. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the threshold value. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a trained setting of a bead template, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().

Aurora Imaging Library typically bases trained values on their related training values and modifies them accordingly when you call MbeadTrain(). For example, Aurora Imaging Library adjusts the specified spacing for the search boxes (M_TRAINING_BOX_SPACING) to achieve an even distribution of search boxes along the template's path (M_TRAINED_BOX_SPACING). If you do not retrain the templates, the trained value represents the value Aurora Imaging Library uses for verification.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a trained setting of a bead template																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_TRAINED_BOX_HEIGHT +		Inquires the trained height of the search boxes. Aurora Imaging Library bases this value on M_TRAINING_BOX_HEIGHT and internally modifies it during the training phase to achieve the best possible trained height.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the trained height of the search boxes.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINED_BOX_SPACING +		Inquires the trained distance between the search boxes in which to establish the trained points. Aurora Imaging Library bases this value on M_TRAINING_BOX_SPACING and internally modifies it during the training phase to achieve an even distribution of trained search boxes.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the trained distance between the search boxes.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_BOX_WIDTH +		Inquires the trained width of the search boxes. Aurora Imaging Library bases this value on a variety of settings, such as M_TRAINING_BOX_WIDTH, to achieve the best possible trained width.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the trained width of the search boxes.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_INDIVIDUAL_WIDTH_NOMINAL +		Inquires the trained width of the template at the position of each individual trained point. Aurora Imaging Library bases this value on a variety of settings, such as M_WIDTH_..., to achieve the best possible trained widths.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the trained width for each trained point.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINED_POSITION_X +		Inquires the trained X-position of the trained points. Aurora Imaging Library bases this value on a variety of settings, such as M_TRAINING_PATH, to achieve the best possible trained positions.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the trained X-position for each trained point.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_POSITION_Y +		Inquires the trained Y-position of the trained points. Aurora Imaging Library bases this value on a variety of settings, such as M_TRAINING_PATH, to achieve the best possible trained positions.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the trained Y-position for each trained point.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINED_SIZE +		Inquires the number of trained points. Aurora Imaging Library considers the number of trained points in a template to be its size.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of trained points.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRAINED_WIDTH_NOMINAL +		Inquires the trained nominal width of the template. Aurora Imaging Library bases this value on a variety of settings, such as M_WIDTH_NOMINAL_MODE, to achieve the best possible trained nominal width.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the trained nominal width of the bead.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_INDIVIDUAL_WIDTH_NOMINAL; M_POSITION_X; M_POSITION_Y; M_TRAINED_INDIVIDUAL_WIDTH_NOMINAL; M_TRAINED_POSITION_X; M_TRAINED_POSITION_Y.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
InquireType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_NB_ELEMENTS		Retrieves the required array size (number of elements) to store the returned values.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a bead template setting for the verification phase, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a bead template setting for the verification phase																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ACCEPTANCE +		Inquires the acceptance level for the score of the template's corresponding measured bead. SET															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 100.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the acceptance level, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ANGLE_ACCURACY_MAX_DEVIATION +		Inquires the maximum angular deviation from the automatically established angle with which to verify the width of the template's measured bead. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 °.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 180.0	Specifies the angular deviation, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BOX_WIDTH_MARGIN +		Inquires the additional width for the search boxes used to verify the template's measured bead. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the additional width used for verification, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAIL_WARNING_OFFSET +		Inquires the additional width specified for the search box where a measured point can be found, but will have a failed status. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the width, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_MAX_LENGTH +		Inquires the maximum allowable length of one single gap, for the template's measured bead to have a passing status. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_GAP_MAX_LENGTH has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the maximum gap length, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GAP_TOLERANCE +		Inquires the percentage of the total allowable gaps (all gaps added together), for the template's measured bead to have a passing status. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0 %.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the total allowable gaps, as a percentage. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_INTENSITY_DELTA_NEG +		Inquires the lowest pixel intensity that is considered valid for the template's measured bead, relative to M_INTENSITY_NOMINAL. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the lowest valid pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_DELTA_POS +		Inquires the highest pixel intensity that is considered valid for the template's measured bead, relative to M_INTENSITY_NOMINAL. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the highest valid pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTENSITY_NOMINAL +		Inquires the nominal pixel intensity of the template's measured bead. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the nominal pixel intensity.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_MAX +		Inquires the maximum allowable distance between the position of the trained points in the template, and the position of the measured points in the target, for the measured bead to have a passing status. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_OFFSET_MAX has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the allowable offset, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_DELTA_NEG +		Inquires the valid delta negative tolerance between the width of the measured bead and the width of the template, for the measured bead to have a passing status. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that M_WIDTH_DELTA_NEG has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the tolerance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDTH_DELTA_POS +		Inquires the valid delta positive tolerance between the width of the measured bead and the width of the template, for the measured bead to have a passing status. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that M_WIDTH_DELTA_POS has no effect.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the tolerance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global bead template setting, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL(), unless otherwise specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a global setting of a bead template																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_BEAD_TYPE +		Inquires the type of the bead. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BEAD_EDGE	Specifies a bead template consisting of an edge. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BEAD_STRIPE	Specifies a bead template consisting of a stripe. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_VALUE +		Inquires the index associated with the template. In this case, you must set the LabelOrIndex parameter to the label of a template.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the template label you specified is not valid (not associated with a template).	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the index value associated with the specified template label.	c		e	f	g	a		i		j	k	l	m	n	b

M_LABEL_VALUE +		Inquires the label that Aurora Imaging Library automatically associated with the template when it was added to the context. In this case, you must set the LabelOrIndex parameter to the index of a template.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the label.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a bead context, the InquireType parameter can be set to one of the following values. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a global setting of a bead context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ASSOCIATED_CALIBRATION +		Inquires the camera calibration context associated with the training image. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that there is no camera calibration context that has been associated with the training image using M_ASSOCIATED_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	Calibration context identifier	Specifies the camera calibration context that is associated with the training image.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context are modified. Although you cannot identify the modification counter's contents, you can compare them throughout your application to know if the context has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_TEMPLATES +	Inquires the number of templates in the bead context. To add and remove templates, use MbeadTemplate().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of templates.	c		e	f	g	a		i		j	k	l	m	n	b
M_STATUS +	Inquires the training status of the bead context. Aurora Imaging Library establishes the status after the last call to MbeadTrain() (using the most current settings of the bead module).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPLETE	Specifies that the context is completely trained. This indicates that each template in the context is trained.	c		e	f	g	a		i		j	k	l	m	n	b
M_NOT_TRAINED	Specifies that the context is not completely trained. This indicates that there are one or more templates in the context that are not trained.	c		e	f	g	a		i		j	k	l	m	n	b

M_PARTIAL	Specifies that the context is partially trained. This indicates that all templates in the context are either partially trained, or both partially and completely trained. M_PARTIAL can also indicate that one or more sections of one or more templates were not found in the training image.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_IMAGE_SIZE_X +	Inquires the width of the internally saved training image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that there is no training image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the training image.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_IMAGE_SIZE_Y +	Inquires the height of the internally saved training image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that there is no training image.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the training image.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRAINING_IMAGE_TYPE +	Inquires the data type and data depth of the internally saved training image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no training image.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + 8	Specifies that the training image is 8-bit unsigned.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about a global setting of a bead context or bead result buffer, the InquireType parameter can be set to the following value. When inquiring about a bead context, set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to M_CONTEXT. When inquiring about a bead result buffer, set the ContextOrResultBeadId parameter to the identifier of a bead result buffer, and

the LabelOrIndex parameter to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about a global setting of a bead context or bead result buffer																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_OWNER_SYSTEM +		Inquires the identifier of the system on which either the bead context or bead result buffer has been allocated. SET SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about settings for operations with MbeadGetNeighbors(), the InquireType parameter can be set to the following value. In this case, you must set the ContextOrResultBeadId parameter to the identifier of a bead context, and the LabelOrIndex parameter to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about settings for operations with MbeadGetNeighbors()																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_CLOSEST_POINT_MAX_DISTANCE +		Inquires the maximum distance specified for the M_CLOSEST_... operations in MbeadGetNeighbors(). SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that the maximum distance is infinite.	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies the maximum distance, relative to the input coordinate system specified using M_TEMPLATE_INPUT_UNITS.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- To inquire about a bead template setting for the training phase
- To inquire about a bead template setting for the training and verification phases
- To inquire about a trained setting of a bead template
- To inquire about a bead template setting for the verification phase
- To inquire about a global setting of a bead template
- To inquire about a global setting of a bead context
- To inquire about a global setting of a bead context or bead result buffer
- To inquire about settings for operations with MbeadGetNeighbors()

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire about the default value of an inquire type																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- To inquire about a bead template setting for the training phase
- To inquire about a bead template setting for the training and verification phases
- To inquire about a trained setting of a bead template
- To inquire about a bead template setting for the verification phase
- To inquire about a global setting of a bead template
- To inquire about a global setting of a bead context
- To inquire about a global setting of a bead context or bead result buffer
- To inquire about settings for operations with MbeadGetNeighbors()

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP

To inquire whether an inquire type is supported																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_HAS_DEFAULT	Inquires whether the specified inquire type has a default value.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED	Inquires whether the specified inquire type is supported for the bead context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- To inquire about a bead template setting for the training phase
- To inquire about a bead template setting for the training and verification phases
- To inquire about a trained setting of a bead template
- To inquire about a bead template setting for the verification phase
- To inquire about a global setting of a bead template
- To inquire about a global setting of a bead context
- To inquire about a global setting of a bead context or bead result buffer
- To inquire about settings for operations with MbeadGetNeighbors()

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

To specify the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*. Note that M_TYPE_AIL_ID should only be used with M_OWNER_SYSTEM.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.

DLL	Requires ail11.dll; ailbead11.dll.

MbeadRestore

Synopsis

Restore a bead context from disk.

Syntax

```
AIL_ID MbeadRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextBeadIdPtr     //out
)
```

Description

This function restores a bead context that was previously saved to a file, using MbeadSave() or MbeadStream(). This function restores all of the bead context's settings that were in effect when it was saved. If the bead context was trained when it was saved, the training information will be restored. You can use MbeadInquire() with M_STATUS to inquire the training status of the bead context after calling this function. If the training status is M_COMPLETE, the training information was restored; otherwise, you must call MbeadTrain() before performing a verification with MbeadVerify().

If you had associated a camera calibration context with the training image before saving, the camera calibration context is automatically restored from the same file as the bead context. The camera calibration cannot be managed independently from the bead context. When the bead context is freed, the camera calibration is automatically freed as well.

When the restored bead context is no longer required, release it using MbeadFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the bead context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the bead context. The function handles (internally) the opening and closing of the file.

This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, bead context files have a MBEAD file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the identifier of the system on which to restore the bead context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextBeadIdPtr

Specifies the address of the variable in which to write the bead context identifier or specifies the data type that the function should use to return the bead context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated bead context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated bead context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BEAD_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the bead context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored bead context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the bead context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BEAD_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadSave

Synopsis

Save a bead context to a file.

Syntax

```
void MbeadSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextBeadId,        //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated bead context to disk, including the current settings of each template, training information, and the camera calibration context associated with the training image, if any. This information can be reloaded, using MbeadRestore() or MbeadStream().

Parameters

FileName

Specifies the name and path of the file in which to save the bead context. It is recommended that you use the MBEAD file extension for easier use with other Aurora Imaging software products. The function internally handles the opening and closing of the file. If this file already exists, it will be overwritten.

This parameter should be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, bead context files have a MBEAD file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextBeadId

Specifies the identifier of the bead context to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadStream

Synopsis

Load, restore, or save an Aurora Imaging Library bead context from/to a file or memory stream.

Syntax

```
void MbeadStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextBeadIdPtr,      //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a bead context from/to a file or memory stream.

To inquire the number of bytes necessary to save a bead context, you should first call this function (**MbeadStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MbeadSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated bead context (and the templates therein) is saved, including training information and the camera calibration context associated with the training image, if any.

All of the bead context's settings that were in effect when the bead context was saved will be restored. If the bead context was trained when it was saved, the training information will be restored. You can use MbeadInquire() with M_STATUS to inquire the training status of the bead context after calling this function. If the training status is M_COMPLETE, the training information was restored; otherwise, you must call MbeadTrain() before performing a verification with MbeadVerify(). If you had associated a training image with a camera calibration context, the camera calibration context is automatically saved to/restored from the same file as the bead context. The camera calibration cannot be managed independently from the bead context. When the bead context is freed, the camera calibration is automatically freed as well.

Using **MbeadStream**(), you can choose to save a backwards-compatible version of the bead context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore bead contexts saved using Aurora Imaging Library version 9.0 PP2 or above. Settings that do not exist in the lower version will be filled with default values when the bead context is loaded or restored. Note that only bead contexts saved using version 11.0 automatically save the training and camera calibration information.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the bead context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the bead context. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the bead context. When performing an M_LOAD or M_RESTORE operation, this parameter must be set to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextBeadIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the bead context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the bead context, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> The size of the bead context varies depending on the Aurora Imaging Library version that you specify.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextBeadIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	MemPtrOrFileName - Possible values passed or returned																
	SysId																
	Version																
	ContextBeadIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE		Inquires the number of bytes required to save a bead context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
	MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version	Specifies the Aurora Imaging Library version with which the bead context must be compatible when saved. The size of a bead context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextBeadIdPtr		Specifies the address of the variable containing the identifier of the bead context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD			Loads the content of a specified file or memory stream into a previously allocated bead context.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Bead contexts typically have a MBEAD file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextBeadIdPtr		Specifies the address of the variable containing the identifier of the bead context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE		Restores a bead context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Bead contexts typically have a MBEAD file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		Specifies the system on which to allocate the bead context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

ContextBeadIdPtr	Specifies the address of the variable in which to write the identifier of the bead context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a bead context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Aurora Imaging software products, when saving a bead context to a file, use the MBEAD file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

	Version	Specifies the Aurora Imaging Library version with which the bead context must be compatible when saved. Note that the bead context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextBeadIdPtr	Specifies the address of the variable containing the identifier of the bead context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadTemplate

Synopsis

Add, delete, or modify a bead template.

Syntax

```
void MbeadTemplate(
    AIL_ID ContextBeadId,             //out
    AIL_INT64 Operation,              //in
    AIL_INT64 BeadType,               //in
    AIL_INT LabelOrIndex,             //in
    AIL_INT GraListIdOrSizeOfArray,   //in
    const AIL_DOUBLE *FirstArrayPtr,  //in
    const AIL_DOUBLE *SecondArrayPtr, //in
    const AIL_INT *ThirdArrayPtr,     //in
    AIL_INT64 ControlFlag             //in
)
```

Description

This function adds a new bead template to a bead context. This function also performs operations on templates already in a context, such as deleting them, modifying them, or adding, deleting, or modifying their vertices.

By default, Aurora Imaging Library interprets certain input settings for a template in pixel units (such as position and dimension). To specify that Aurora Imaging Library should interpret these settings in world units, use MbeadControl() with M_TEMPLATE_INPUT_UNITS set to M_WORLD.

Whenever you add, delete, or modify a bead template, you must train it using MbeadTrain(). Once you train the template, you can use it to verify beads with MbeadVerify().

Parameters

ContextBeadId

Specifies the identifier of a bead context. The bead context must have been previously allocated on the required system using MbeadAlloc().

Operation

Specifies to add a new bead template to a bead context, or specifies another operation to perform with a template that is already in a context.

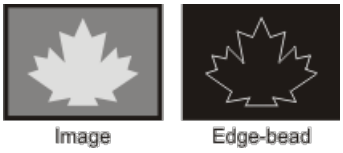
See the Parameter associations section for possible values that can be specified.

BeadType INQ

Specifies the type of the new bead template to add to a bead context, or specifies that the type information is not required.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the bead type																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default type. When adding a new bead template to a bead context, M_DEFAULT is the same as M_BEAD_STRIPE. When performing an operation with a template that is already in a context, M_DEFAULT indicates that the type information is not required. This is the only possible setting for these operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_BEAD_EDGE	Specifies to add a new bead template consisting of an edge. In this case, the template represents an outline of a bead delineated by a boundary established from intensity transitions in an image, such as the skeletal border of an object.  By default, Aurora Imaging Library assumes the foreground of the edge-bead is white and is on the counter-clockwise side of the path direction, while the background is assumed to be black and on the clockwise side of the edge-bead. To modify how Aurora Imaging Library establishes the foreground, use MbeadControl() with M_FOREGROUND_VALUE. You can only specify M_BEAD_EDGE when adding a new bead template to a bead context.	c		e	f	g	a		i		j	k	l	m	n	b
M_BEAD_STRIPE	Specifies to add a new bead template consisting of a stripe. In this case, the template represents a bead that has two edges and a width, such as a strip of some malleable material (for example, glue or lead). By default, Aurora Imaging Library assumes the stripe-bead is white and is on a black background. By default, Aurora Imaging Library also assumes that the bead's nominal width is consistent and is automatically learned from the training phase. To modify how Aurora Imaging Library establishes the foreground and width, use MbeadControl() with M_FOREGROUND_VALUE and M_WIDTH_NOMINAL_MODE. You can only specify M_BEAD_STRIPE when adding a new bead template to a bead context.	c		e	f	g	a		i		j	k	l	m	n	b

LabelOrIndex

Specifies a unique label for a new bead template to add to a bead context, or specifies the index or label of

a template in a context (or all templates in a context) with which to perform the operation. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a bead template																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TEMPLATE_INDEX(AIL_INT Index)	Specifies the index of the bead template.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Index Specifies the index, which must be greater than or equal to 0.															
M_TEMPLATE_LABEL(AIL_INT Label)	Specifies the label of the bead template.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Label Specifies the label, which must be greater than 0.															
M_ALL	Specifies all bead templates.	c		e	f	g	a		i		j	k	l	m	n	b

GraListIdOrSizeOfArray

Specifies information required for the operation. If this information is not needed, set this parameter to **M_NULL**.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the FirstArrayPtr, SecondArrayPtr, or ThirdArrayPtr parameter.

See the Parameter associations section for possible values that can be specified.

FirstArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]

Specifies an attribute of the operation. Its definition is dependent on the Operation parameter setting. If the operation does not require this information, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

SecondArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]

Specifies an attribute of the operation. Its definition is dependent on the Operation parameter setting. If the operation does not require this information, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

ThirdArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>]
- array of type AIL_INT [optionally, in C++: a reference to a constant std::vector<AIL_INT>]

Specifies an attribute of the operation. Its definition is dependent on the Operation parameter setting. If the operation does not require this information, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

The tables below list possible values for the **Operation**, **GraListIdOrSizeOfArray**, **FirstArrayPtr**, **SecondArrayPtr**, and **ThirdArrayPtr** parameters.

- For adding a new bead template to a bead context
- For performing an operation with a bead template in a bead context

Set unused parameters to **M_NULL**.

The value in the table below is for adding a new bead template to a bead context. In this case, you must set the LabelOrIndex parameter to M_TEMPLATE_LABEL() and specify a unique label value for the new template.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For adding a new bead template to a bead context																	
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	GraListIdOrSizeOfArray																
	FirstArrayPtr																
	SecondArrayPtr																
	ThirdArrayPtr																

M_ADD	Adds a new bead template, based on the specified vertices or a fixed circle or segment, to a bead context. A template must have a path and a width. To establish the path of the template, use MbeadControl() with M_TRAINING_PATH. The template's default path (M_POLYLINE_SEED) follows a polyline that Aurora Imaging Library establishes using the specified vertices (FirstArrayPtr and SecondArrayPtr) and then refining them according to a training image (MbeadTrain()). To specify a polyline path with a trained position that exactly follows the specified vertices, use M_POLYLINE. In either case, the order in which you specify the vertices establishes the path's shape and direction. You can also specify that the template's path follows a fixed segment or circle, which you must define using MbeadControl() with M_TEMPLATE_CIRCLE_... or M_TEMPLATE_SEGMENT_.... In this case you should not set any vertices. To establish the width of the template, use MbeadControl() with M_WIDTH_NOMINAL_MODE. By default (M_AUTO_UNIFORM), Aurora Imaging Library uses a training image to establish an average bead width. To use a training image to establish a unique width at the position of each trained point, use M_AUTO_CONTINUOUS. You can also explicitly specify the template's width using M_USER_DEFINED. You can modify a template's vertices or width after you have set them (for example, using M_INSERT or M_SET_WIDTH_NOMINAL); keep this in mind since Aurora Imaging Library bases the template's path and width using the most current settings when you call MbeadTrain().															
GraListIdOrSizeOfArray	Sets the size of the first and second source arrays (FirstArrayPtr and SecondArrayPtr). If the template's path follows a segment or circle, set this parameter to **M_NULL**. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	FirstArrayPtr	Sets the address of the array containing the X-coordinates of the template's vertices. You must specify at least 2 vertices. If the template's path follows a segment or circle, set this parameter to **M_NULL**. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	SecondArrayPtr	Sets the address of the array containing the Y-coordinates of the bead template's vertices. You must specify at least 2 vertices. If the template's path follows a segment or circle, set this parameter to **M_NULL**. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_ADD_FROM_GRAPHIC_LIST	Adds a new bead template, based on the vertices of a polyline (or polygon) from a 2D graphics list, to a bead context. The 2D graphics list must contain one polyline (or polygon) graphic, added with MgraLines() or MgraInteractive(). Aurora Imaging Library considers the specified vertices (MgraLines() or MgraInteractive()) as the template's vertices. Note that a polygon is simply a polyline that Aurora Imaging Library automatically closes by connecting the final vertex to the first with a straight line. Any operation or setting that applies to polylines also applies to polygons. A template must have a path and a width. To establish the path of the template, use MbeadControl() with M_TRAINING_PATH. The template's default path (M_POLYLINE_SEED) follows a polyline that Aurora Imaging Library establishes using the specified vertices (MgraLines() or MgraInteractive()) and then refining them according to a training image (MbeadTrain()). To specify a polyline path with a trained position that exactly follows the specified vertices, use M_POLYLINE. In either case, the order in which you specify the vertices (MgraLines()) establishes the path's shape and direction. To establish the width of the template, use MbeadControl() with M_WIDTH_NOMINAL_MODE. By default (M_AUTO_UNIFORM), Aurora Imaging Library uses a training image to establish an average bead width. To use a training image to establish a unique width at the position of each trained point, use M_AUTO_CONTINUOUS. You can also explicitly specify the template's width using M_USER_DEFINED. You can modify a template's vertices or width after you have set them (for example, using M_INSERT or M_SET_WIDTH_NOMINAL); keep this in mind since Aurora Imaging Library bases the template's path and width using the most current settings when you call MbeadTrain().	c		e	f	g	a		i		j	k	l	m	n	b
GraListIdOrSizeOfArray	Sets the identifier of a 2D graphics list.	c		e	f	g	a		i		j	k	l	m	n	b

The values in the table below are for performing an operation with a bead template in a bead context. Unless otherwise specified, you can only perform these operations if the template's path follows a polyline, and you must set the LabelOrIndex parameter to a specific template, using either M_TEMPLATE_INDEX() or M_TEMPLATE_LABEL().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

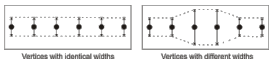
For performing an operation with a bead template in a bead context

Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
GraListIdOrSizeOfArray																
FirstArrayPtr																
SecondArrayPtr																
ThirdArrayPtr																
M_DELETE	Deletes the specified vertices from the specified template, or deletes the specified template. You can delete templates whose path follows a polyline, circle, or segment; however you can only delete vertices from a template if its path follows a polyline.															
GraListIdOrSizeOfArray	Sets whether to delete vertices or templates.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to delete the templates indicated by the LabelOrIndex parameter. To delete all templates, set the LabelOrIndex parameter to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to delete all vertices from the specified template. In this case, you cannot set the LabelOrIndex parameter to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the size of the array (ThirdArrayPtr) containing the vertices to delete from the specified bead template. In this case, you cannot set the LabelOrIndex parameter to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b

ThirdArrayPtr	Sets the address of the array containing the indices of the vertices to delete, or specifies that this parameter is ignored.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies to ignore this parameter. You must set this parameter to M_NULL when you are deleting templates (GraListIdOrSizeOfArray is set to M_NULL) or when deleting all vertices from a template (GraListIdOrSizeOfArray is set to M_ALL).	c		e	f	g	a		i		j	k	l	m	n	b
ArrayAddress	Specifies the address of the array that indicates the vertices to delete. The array must have the same size as specified using GraListIdOrSizeOfArray.	c		e	f	g	a		i		j	k	l	m	n	b
M_INSERT	Inserts the specified vertices into the specified bead template. INQ	c		e	f	g	a		i		j	k	l	m	n	b
GraListIdOrSizeOfArray	Sets the size of the first and second source arrays (FirstArrayPtr and SecondArrayPtr).	c		e	f	g	a		i		j	k	l	m	n	b
FirstArrayPtr	Sets the address of the array containing the X-coordinates of the vertices to insert into the bead template. INQ	c		e	f	g	a		i		j	k	l	m	n	b
SecondArrayPtr	Sets the address of the array containing the Y-coordinates of the vertices to insert into the bead template. INQ	c		e	f	g	a		i		j	k	l	m	n	b
ThirdArrayPtr	Sets the address of the variable containing the place to insert the vertices. This variable must hold one of the following values:	c		e	f	g	a		i		j	k	l	m	n	b

		M_BEGIN	Specifies that the vertices will be inserted before the bead template's first vertex.	c		e	f	g	a		i		j	k	l	m	n	b
		M_END	Specifies that the vertices will be inserted after the bead template's last vertex.	c		e	f	g	a		i		j	k	l	m	n	b
		-1 <= *Value* < *TotalNumberOfVertices*	Specifies an index value representing the position in the template after which to insert the specified vertices. Since the index of the template's first vertex is 0, you must set a value between negative one (inserts vertices before the bead template's first vertex) and the total number of vertices in the template minus one (inserts vertices after the bead template's last vertex).	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLACE			Replaces the specified vertices of the specified bead template.	c		e	f	g	a		i		j	k	l	m	n	b
	GraListIdOrSizeOfArray		Sets the size of the first, second, and third source arrays (FirstArrayPtr, SecondArrayPtr, and ThirdArrayPtr).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstArrayPtr		Sets the address of the array containing the X-coordinates of the vertices that will replace the specified vertices of the bead template (ThirdArrayPtr).	c		e	f	g	a		i		j	k	l	m	n	b
	SecondArrayPtr		Sets the address of the array containing the Y-coordinates of the vertices that will replace the specified vertices of the bead template (ThirdArrayPtr).	c		e	f	g	a		i		j	k	l	m	n	b

	ThirdArrayPtr	Sets the address of the array containing the indices of the vertices in the bead template that will be replaced by the new vertices.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATE_POINTS		Rotates the specified bead template around its origin (0,0). This operation supports templates with any type of path.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstArrayPtr	Sets the address of the variable containing the angle by which to rotate the bead template. Positive angles indicate a counter-clockwise rotation.	c		e	f	g	a		i		j	k	l	m	n	b
M_SCALE_POINTS		Scales the specified bead template. This operation supports templates with any type of path.	c		e	f	g	a		i		j	k	l	m	n	b
	FirstArrayPtr	Sets the address of the variable containing the factor by which to scale the bead template. The scale factor must be greater than zero.	c		e	f	g	a		i		j	k	l	m	n	b

M_SET_WIDTH_NOMINAL		Specifies the expected width of the specified bead template. When using M_SET_WIDTH_NOMINAL, Aurora Imaging Library adjusts the template's nominal width (MbeadControl() with M_WIDTH_NOMINAL) according to the specified width value (FirstArrayPtr) at each specified vertex of the template (ThirdArrayPtr). You should only set the width when it differs from the nominal width of the bead. Aurora Imaging Library linearly interpolates the width between consecutive vertices.  Vertices with identical widths Vertices with different widths For M_SET_WIDTH_NOMINAL to have an effect, you must specify that the bead's nominal width is user-defined, using MbeadControl() with M_WIDTH_NOMINAL_MODE set to M_USER_DEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
	GraListIdOrSizeOfArray	Sets the size of the required source arrays (FirstArrayPtr and ThirdArrayPtr).	c		e	f	g	a		i		j	k	l	m	n	b
	FirstArrayPtr	Sets the address of the array containing the width of the corresponding vertices specified with ThirdArrayPtr. To reset a vertex's width back to the nominal width (M_WIDTH_NOMINAL), set its width in this array to **M_RESET_WIDTH**. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	ThirdArrayPtr	Sets the address of the array containing the indices of the vertices for which to set the width.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRANSLATE_POINTS	Moves the specified bead template. This operation supports templates with any type of path.	c		e	f	g	a		i		j	k	l	m	n	b
FirstArrayPtr	Sets the address of the variable containing the amount by which to displace the bead template, in the X-direction.	c		e	f	g	a		i		j	k	l	m	n	b
SecondArrayPtr	Sets the address of the variable containing the amount by which to displace the bead template, in the Y-direction.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadTrain

Synopsis

Train the templates in a bead context.

Syntax

```
void MbeadTrain(
    AIL_ID ContextBeadId,      //in
    AIL_ID TrainingImageBufId, //in
    AIL_INT64 ControlFlag      //in
)
```

Description

This function trains the templates for a verification operation and sets internal processing settings to optimize future calculations for speed and robustness. This function can also internally save or free a training image (TrainingImageBufId).

You must call this function before your first call to MbeadVerify(). Certain modifications to a bead context, such as adding, deleting, or modifying templates with MbeadTemplate(), or changing some control settings with MbeadControl(), require you to retrain the templates before any subsequent call to MbeadVerify().

To inquire the training status of the templates in the context, use MbeadInquire() with M_STATUS. Typically you should not call MbeadVerify() until all the templates in the context have been completely trained. You might have to modify your settings and retrain the templates more than once before you can perform a successful verification. To draw the templates in the context (trained or untrained), use MbeadDraw().

To train the templates, Aurora Imaging Library must establish their trained points, which are based on the most current settings, such as the spacing between the expected position of the trained points, the template's path (polyline, circle, or segment), and width. To train the templates, Aurora Imaging Library also uses the training image, if it is required by the template's settings.

By default, you must use a training image, since the default path a template follows is a polyline that is refined by the training image (MbeadControl() with M_TRAINING_PATH set to M_DEFAULT or M_POLYLINE_SEED). If a template follows a path that is a fixed polyline, circle, or segment (M_POLYLINE, M_CIRCLE, or M_SEGMENT), the path's position will not be refined, even if you specify a training image. Similarly, Aurora Imaging Library uses the training image by default to establish the width of the template's path (MbeadControl() with M_WIDTH_NOMINAL_MODE set to M_DEFAULT or M_AUTO_...). If the nominal width is explicitly set, a training image is not required and is ignored if provided.

If you have associated the training image with a camera calibration context, you can specify that certain input settings for the training phase be interpreted in world units. To do so, use MbeadControl() with M_TEMPLATE_INPUT_UNITS and/or M_TRAINING_BOX_INPUT_UNITS set to M_WORLD. To use a calibrated training image, you can either pass one to this function, or you can specify the camera calibration context to associate to the training image, using MbeadControl() with M_ASSOCIATED_CALIBRATION. If both camera calibrations are available, Aurora Imaging Library ignores the camera calibration set with M_ASSOCIATED_CALIBRATION. Note that if Aurora Imaging Library is interpreting input settings for the training phase in world units, but you don't use a calibrated training image, **MbeadTrain**() will generate an error.

If you are specifying pixel units, make sure the pixel sizes are consistent across all camera calibrations (for example, pixel sizes should be the same in the training and verification images). When using different camera calibration contexts, it is recommended that you use world units.

Parameters

ContextBeadId

Specifies the identifier of the bead context that contains the templates to train. The bead context must have been previously allocated on the required system using MbeadAlloc().

TrainingImageBufId

Specifies the identifier of the image buffer containing the training image with which to train the templates, or specifies to train the templates without a training image (or with a previously saved training image).

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the source																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies no training image will be used to train the templates, unless the context contains a training image from previous training. In this case, Aurora Imaging Library uses the saved training image to train the templates even if you specify M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
Image buffer identifier	Specifies the identifier of an image buffer containing the training image Aurora Imaging Library will use to train the templates. The image buffer must be a 1-band 8-bit unsigned buffer. All templates in the context use the same training image. Note that the training image will be internally saved. If the context contains a training image from previous training, it will be freed from memory and the newly specified image will be used to train the templates. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. To train the templates, Aurora Imaging Library extracts beads from the training image using processes based on the Aurora Imaging Library Measurement module. Such processes use a one-dimensional analysis of differences in pixel intensities. For more information about this type of edge extraction, see the Search algorithm section of Chapter 19: Measurement.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.

MbeadVerify

Synopsis

Verify beads in the target image according to their corresponding template.

Syntax

```
void MbeadVerify(
    AIL_ID ContextBeadId,    //in
    AIL_ID TargetImageBufId, //in
    AIL_ID ResultBeadId,     //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function verifies that beads in the target image conform to their corresponding templates in the context. Results of the measured bead are written in the provided result buffer and can be read using MbeadGetResult(). To retrieve whether a measured bead has passed or failed the specifications of its template, use the M_STATUS result. To draw results, use MbeadDraw().

Before your first call to **MbeadVerify**(), you must train the context using MbeadTrain(). Certain modifications to the bead context, such as adding, deleting, or modifying templates with MbeadTemplate(), or changing some control settings with MbeadControl(), require you to retrain the context before any subsequent call to **MbeadVerify**(). To inquire the training status of the templates in the context, use MbeadInquire() with M_STATUS.

If you have associated the target image with a camera calibration context, you can specify that certain input settings be interpreted in world units. To do so, use MbeadControl() with M_TEMPLATE_INPUT_UNITS set to M_WORLD. Note that if you set this constant to M_WORLD but you don't pass **MbeadVerify**() a calibrated target image, the function will generate an error.

If you are specifying pixel units, make sure the pixel sizes are consistent across all camera calibrations (for example, pixel sizes should be the same in the training and verification images). When using different camera calibration contexts, it is recommended that you use world units.

Parameters

ContextBeadId

Specifies the identifier of the bead context that contains the templates with which to validate the beads in the target image. The bead context must have been previously allocated on the required system using MbeadAlloc().

TargetImageBufId

Specifies the identifier of the target image buffer that contains the beads that must be verified, according the templates contained within the bead context. The image buffer must be a 1-band 8-bit unsigned buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

To perform the verification, Aurora Imaging Library extracts beads from the training image using processes based on the Aurora Imaging Library Measurement module. Such processes use a one-

dimensional analysis of differences in pixel intensities. For more information about this type of edge extraction, see the Search algorithm section of Chapter 19: Measurement.

ResultBeadId

Specifies the identifier of the bead result buffer in which to write the results of the verification. The bead result buffer must have been previously allocated on the required system using MbeadAllocResult().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailbead11.lib.
DLL	Requires ail11.dll; ailbead11.dll.